



LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : M. IKBAL PRATAMA

NIM : 2428240135

Kelas : SI5C

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK/.....

Identitas Praktikum

Nama Mahasiswa	M. IKBAL PRATAMA
NPM	2428240135
Kelas	SI5C
Pertemuan Ke-	2
Topik / Judul Praktikum	Full rest API
Tanggal Pelaksanaan	
Dosen Pengampu	<u>Nur Rachmat, M.Kom</u>

I. Tujuan Praktikum

Tuliskan tujuan praktikum sesuai indikator capaian pembelajaran pada pertemuan terkait (lihat slide materi bagian “Capaian Pembelajaran”).

1. Mahasiswa mampu menunjukkan kemampuan berkomunikasi dan berkolaborasi dalam sebuah kegiatan.

2. Mahasiswa mampu mengembangkan sebuah aplikasi sesuai dengan permasalahan dan kebutuhan.

3. Mahasiswa memiliki kemampuan menerapkan konsep teoritis terkait JSON dan RESTful API yang aman (termasuk autentikasi stateless/JWT) dalam mengembangkan penyimpanan data NoSQL pada arsitektur aplikasi web backend frontend terpisah (decoupled).

II. Dasar Teori

Uraikan secara singkat konsep/teori yang mendasari praktikum ini (maksimal 1 halaman). Sertakan sumber rujukan bila mengutip dari pustaka.

1. Node.js sebagai lingkungan eksekusi

Node.js adalah *runtime* JavaScript yang dibangun di atas mesin V8 milik Google Chrome. Dengan Node.js, JavaScript bisa berjalan di luar browser, yaitu di sisi server. Node.js memakai model *event-driven* dan *non-blocking I/O* dengan satu *event loop*. Server dapat melayani banyak koneksi bersamaan tanpa membuat satu thread per permintaan. Model ini cocok untuk API yang banyak menangani permintaan singkat.

Node Version Manager (NVM) mengelola beberapa versi Node.js dalam satu komputer. Dasar teorinya adalah *dependency and environment management*: setiap project bisa membutuhkan versi runtime yang berbeda, dan versi yang tidak seragam antar anggota tim menimbulkan masalah "jalan di komputer saya, error di komputer Anda". Versi **LTS (Long Term Support)** dipilih karena mendapat dukungan dan perbaikan keamanan dalam jangka panjang, sehingga lebih stabil untuk pengembangan dan produksi.

2. NPM dan manajemen dependensi

NPM (Node Package Manager) adalah pengelola paket bawaan Node.js. Konsep intinya:

- **package.json** adalah *manifest* project. File ini mencatat metadata, skrip, dan dependensi yang dibutuhkan.
 - **package-lock.json** mengunci versi persis semua paket, termasuk dependensi turunannya, agar hasil instalasi *reproducible* di setiap komputer.
 - **dependencies vs devDependencies**: dependensi produksi dibutuhkan aplikasi saat berjalan (mis. express), sedangkan dependensi pengembangan hanya dipakai saat membangun (mis. nodemon, opsi -D).
 - **Semantic Versioning (MAJOR.MINOR.PATCH)** mendasari perintah npm update. Pembaruan hanya dilakukan dalam rentang versi yang diizinkan package.json.
 - **node_modules** dapat dibuat ulang dari package.json, jadi tidak perlu di-commit dan cukup dimasukkan ke .gitignore.
-

3. Express.js sebagai web framework

Express.js adalah *framework* minimalis di atas modul HTTP bawaan Node.js. Ia mengurangi kode *boilerplate* yang harus ditulis. Karena bersifat *unopinionated*, ia tidak memaksakan struktur project tertentu. Empat karakteristik yang disebut materi:

1. **Routing** memetakan kombinasi URL dan HTTP method ke fungsi *handler*.
2. **Middleware** adalah fungsi perantara dalam rantai pemrosesan request. Polanya mirip *chain of responsibility*: setiap fungsi dapat memproses request lalu meneruskannya ke tahap berikutnya.

3. **Ekosistem** berupa ribuan paket pendukung (mis. cors, dotenv).
4. **Ringan dan fleksibel** karena intinya kecil dan fitur tambahan dipasang sesuai kebutuhan.

4. REST dan RESTful API

REST (Representational State Transfer) diperkenalkan Roy Fielding dalam disertasi doktoralnya (2000) sebagai gaya arsitektur untuk sistem terdistribusi berbasis web. REST bukan protokol atau standar, melainkan kumpulan batasan (*constraints*): *client-server*, *stateless*, *cacheable*, *uniform interface*, *layered system*, dan *code on demand* (opsional). API yang mematuhi batasan ini disebut **RESTful**.

Empat prinsip pada slide adalah penyederhanaan yang tepat untuk pembelajaran:

- **Resource-based**: setiap entitas (mahasiswa, produk) diidentifikasi lewat URL, mis. /mahasiswa. Praktik baiknya memakai kata benda, bukan kata kerja.
- **Stateless**: setiap request membawa semua informasi yang dibutuhkan. Server tidak menyimpan status sesi klien, sehingga API lebih mudah diskalakan.
- **Penggunaan HTTP method sesuai semantiknya**, yang dipetakan ke operasi CRUD.
- **Format data seragam**, umumnya JSON. Perlu dicatat bahwa REST sendiri tidak mewajibkan JSON. Ini konvensi yang paling lazim.

5. Protokol HTTP: method, status code, request-response

HTTP bekerja dengan pola *request-response*, yaitu klien mengirim permintaan dan server membalas. Semantiknya dibakukan IETF dalam RFC 9110.

CRUD dan HTTP method

Operasi Method Sifat menurut spesifikasi

Create	POST	Tidak idempoten
Read	GET	Safe dan idempoten
Update	PUT	Idempoten (semantik: mengganti seluruh resource)
Delete	DELETE	Idempoten

Idempoten berarti permintaan yang sama, dikirim berulang, meninggalkan server pada keadaan akhir yang sama. Satu catatan kritis: kode PUT pada materi menggabungkan data lama dengan data baru ({ ...lama, ...req.body }). Secara semantik, itu lebih

mendekati **PATCH** (pembaruan parsial). Untuk tujuan belajar ini wajar, tetapi berguna diketahui kalau Anda menulis pembahasan yang lebih ketat.

Kelompok kode status

- **2xx (berhasil):** 200 OK, 201 Created (setelah POST), 204 No Content (setelah DELETE, tanpa body).
 - **4xx (kesalahan klien):** 400 Bad Request, 404 Not Found.
 - **5xx (kesalahan server):** 500 Internal Server Error.
-

Kode status yang tepat membuat klien dapat menangani hasil secara otomatis tanpa membaca isi pesan. Karena itu "selalu mengembalikan 200" masuk daftar kesalahan umum.

6. Routing dan objek request/response di Express.js

Routing adalah mekanisme menentukan respons aplikasi terhadap kombinasi URL dan HTTP method (`app.get()`, `app.post()`, `app.put()`, `app.delete()`). Route dievaluasi **berurutan dari atas ke bawah** dan yang cocok pertama akan dieksekusi. Inilah dasar teori kesalahan "route statis diletakkan setelah route dinamis": `/mahasiswa/aktif` akan tertangkap oleh `/mahasiswa/:id` sehingga tidak pernah tercapai.

Tiga sumber data dari klien:

- `req.params` berisi bagian dinamis URL (`/mahasiswa/:id`), untuk **mengidentifikasi resource**. Nilainya selalu bertipe *string*, sehingga perlu dikonversi (`parseInt`) sebelum dibandingkan dengan angka.
 - `req.query` berisi *query string* (`?jurusan=SI`), untuk **menyaring atau mengatur tampilan** koleksi.
 - `req.body` berisi data yang dikirim pada badan request, untuk **membawa data** pada POST dan PUT.
-

Objek response: `res.status()` menetapkan kode status, `res.json()` mengirim JSON, dan `res.send()` mengirim teks atau HTML.

Middleware `express.json()` mengurai (*parse*) body ber-format JSON menjadi objek JavaScript. Tanpa middleware ini, `req.body` bernilai `undefined`. Ia harus didaftarkan **sebelum** route yang membaca body, karena middleware dieksekusi sesuai urutan pendaftaran.

7. JSON sebagai format pertukaran data

JSON (JavaScript Object Notation) adalah format teks ringan berbasis pasangan *key-value*, dibakukan dalam RFC 8259 dan ECMA-404. Kelebihannya mudah dibaca manusia, mudah diurai mesin, dan independen dari bahasa pemrograman. Materi menekankan

konsistensi struktur respons: respons sukses membawa data dengan kode 2xx, dan respons gagal membawa pesan yang jelas (mis. { "message": "Data tidak ditemukan" }) dengan kode 4xx atau 5xx. Konsistensi ini adalah kontrak antara server dan klien.

8. Penyimpanan sementara dan identifier

Contoh kode memakai **array di memori** sebagai penyimpanan. Data bersifat *volatile* dan hilang saat server dimatikan, sehingga cocok untuk pembelajaran konsep dan bukan untuk produksi (di produksi dipakai database). Ada satu titik lemah teoretis pada kode POST: `id: mahasiswa.length + 1` dapat menghasilkan **id duplikat** setelah ada data yang dihapus. Prinsip *unique identifier* menuntut id tidak dipakai ulang, misalnya dengan penghitung id terpisah atau, nantinya, *auto-increment* atau UUID di database.

9. Perancangan API dan pengujian

- **Desain API sebelum kode:** endpoint dirancang lebih dulu dalam tabel (method, endpoint, deskripsi, status) seperti pada studi kasus Todo App. Ini praktik *design-first* yang mengurangi kesalahan dan memudahkan kolaborasi tim.
- **Pengujian endpoint** memakai alat seperti Thunder Client atau Postman. Ini *black-box testing*: API diuji dari sisi klien, dengan memverifikasi kode status dan isi respons tanpa melihat kode internal.
- **Kesalahan umum** (lupa `express.json()`, urutan route keliru, lupa mengirim respons sehingga request menggantung, kode status tidak sesuai) semuanya berakar pada konsep di atas: urutan eksekusi middleware dan route, siklus request-response, dan semantik status HTTP.

10. Landasan pedagogis (opsional)

Sub-CPMK 4 berlevel **C3 (Menerapkan)** pada Taksonomi Bloom ranah kognitif. Tugas Project Mandiri menerapkan pembelajaran berbasis proyek: mahasiswa memilih entitas, merancang endpoint, mengimplementasikan, lalu menguji dan mendokumentasikan.

Daftar Pustaka

- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (disertasi doctoral). University of California, Irvine.
- Fielding, R., Nottingham, M., & Reschke, J. (2022). *RFC 9110: HTTP Semantics*. IETF.
- Bray, T. (2017). *RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format*. IETF.
- Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.

- Dokumentasi resmi: expressjs.com, nodejs.org, docs.npmjs.com, dan repositori [nvm-sh/nvm](https://github.com).

Kalau dasar teori ini akan dipakai untuk laporan atau makalah, saya

III. Alat dan Bahan

Sebutkan perangkat lunak/perkakas yang digunakan selama praktikum.

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Versi 24.19
2	Visual Studio Code	Versi terbaru
3	CMD/Comand Promt	windows
4	Github	Versi terbaru

IV. Langkah Kerja

Jelaskan langkah-langkah yang dilakukan selama praktikum secara berurutan.

1.

Yang pertama kami menginstall node js versi terbaru

2.

Setelah itu kami menkonfigurasi nya di CMD Comand Prompt

3.

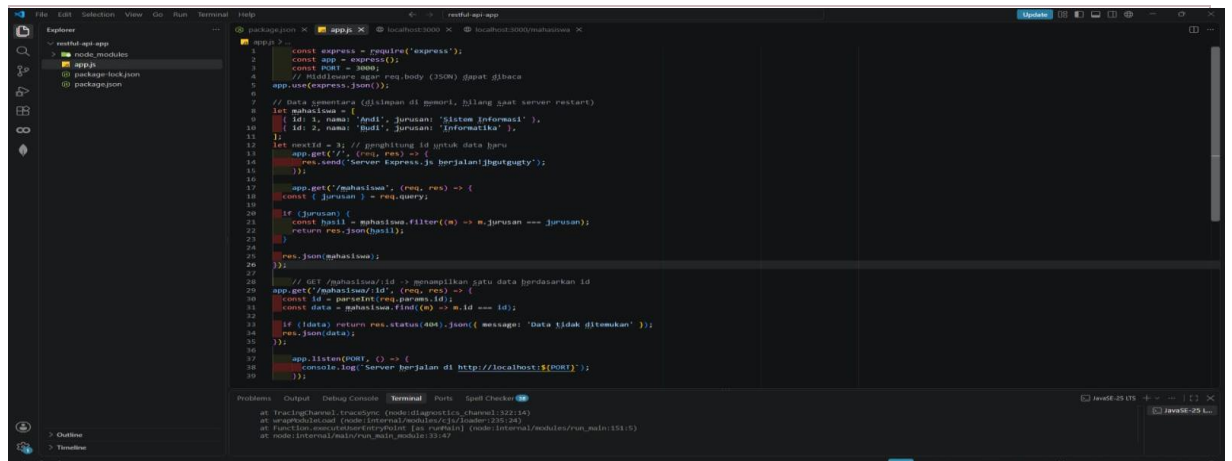
Setelah terkonfigurasi, kami membuat file baru di vs code, disana kami menginstall npm run dev dan memunculkan local host untuk menampilkan hasil output nya

4.

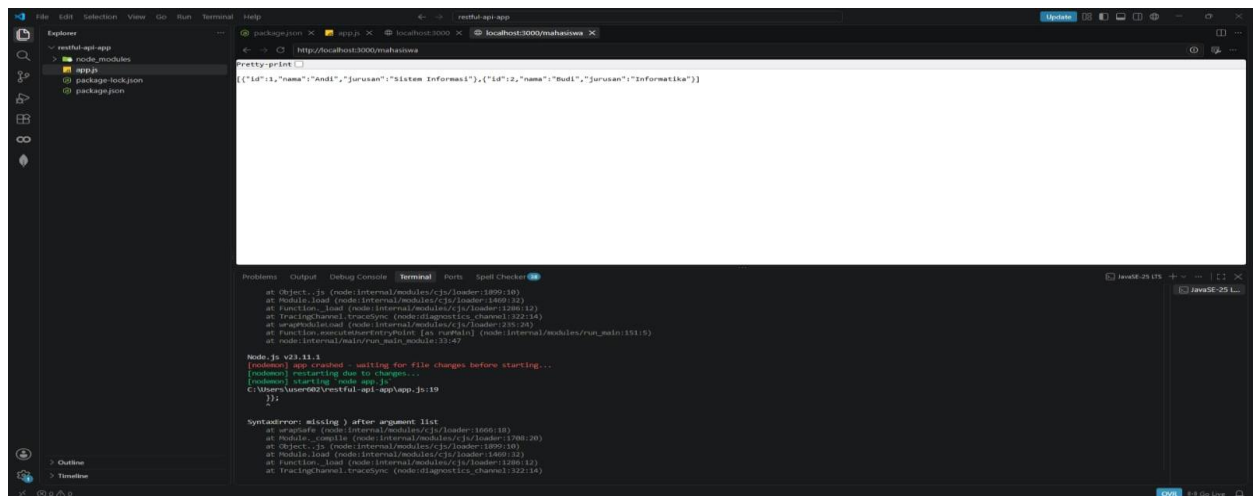
5.

V. Hasil dan Pembahasan

Sertakan tangkapan layar (screenshot) hasil praktikum, disertai penjelasan/pembahasan pada setiap hasil.



Gambar 1. Proses konfigurasi



Gambar 2.hasil ouput nya

Pembahasan:

VI. Kendala dan Solusi

Opsional - isi apabila terdapat kendala teknis selama praktikum.

No.	Kendala	Solusi
1	Agak susah pas konfigurasi	Melihat tutorial di spon
2		

VII. Kesimpulan

Simpulkan hasil praktikum dikaitkan dengan tujuan pada bagian I.

Daftar Pustaka

Tuliskan sumber rujukan yang digunakan (format APA 7th edition).

Lampiran

Tautan Repository GitHub:

Tautan Deployment (jika ada):
